

Proyecto: Grupo 6

Desarrollo Orientado Objetos

Antonia Florencia Sandoval Reyes

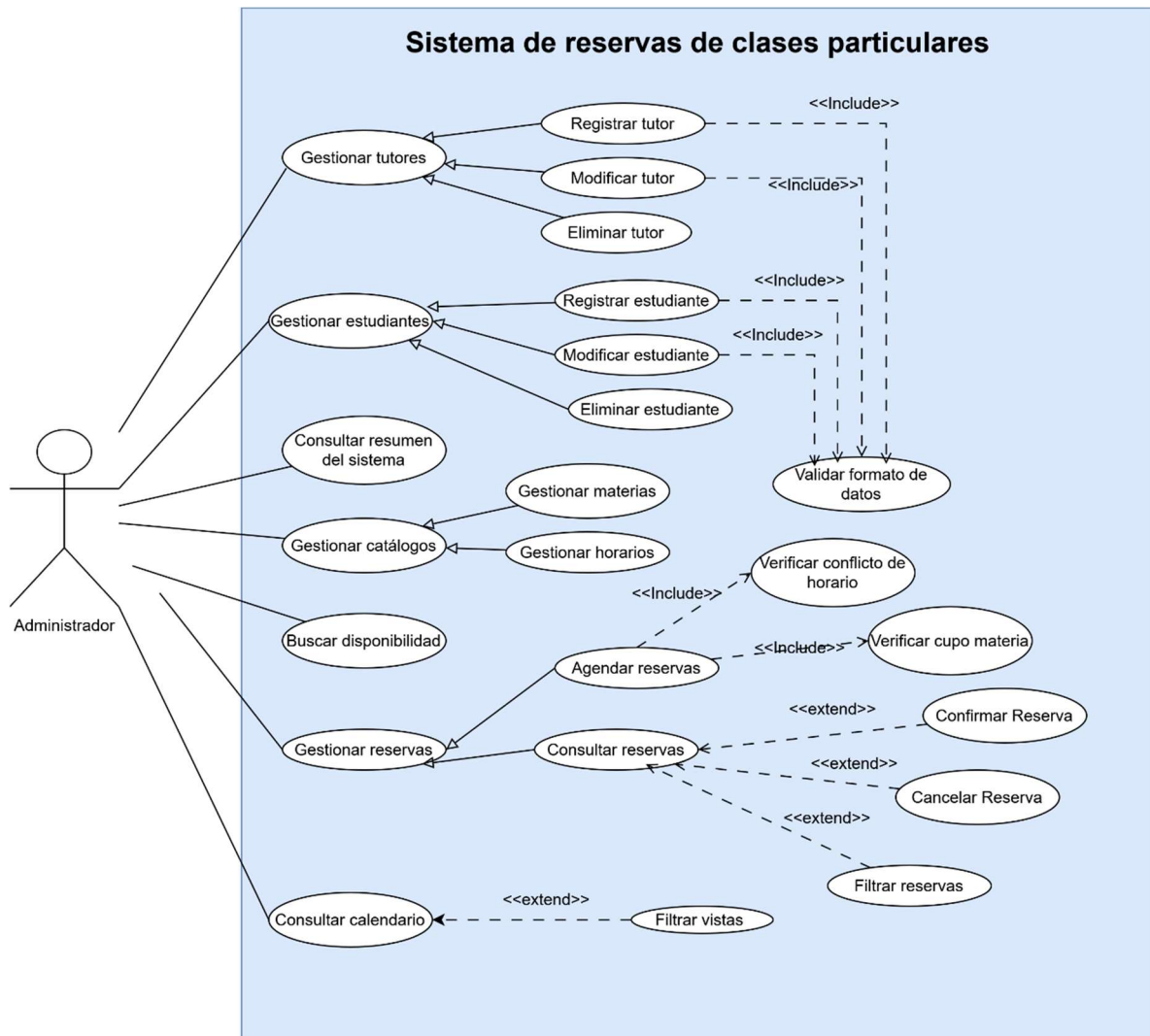
Diego Antonio Gavilan Ramírez

Jorge Andrés Orellana Cuevas

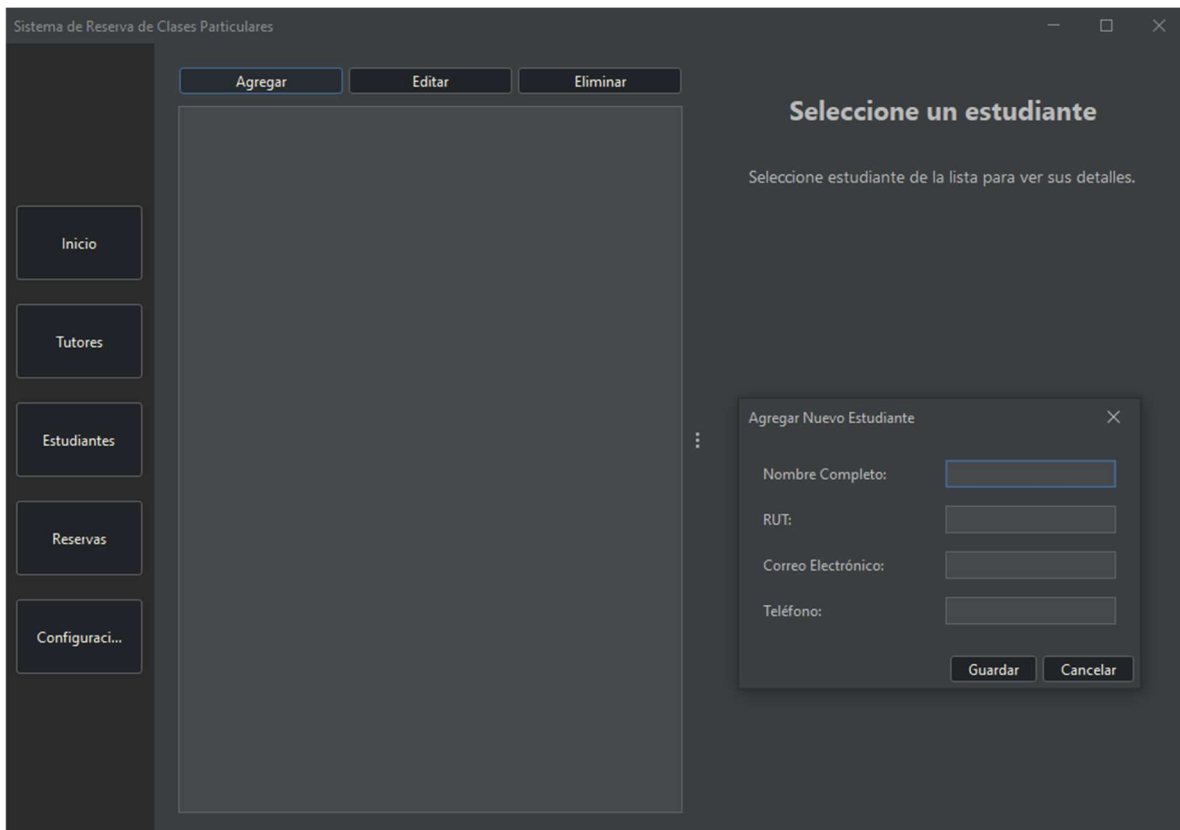
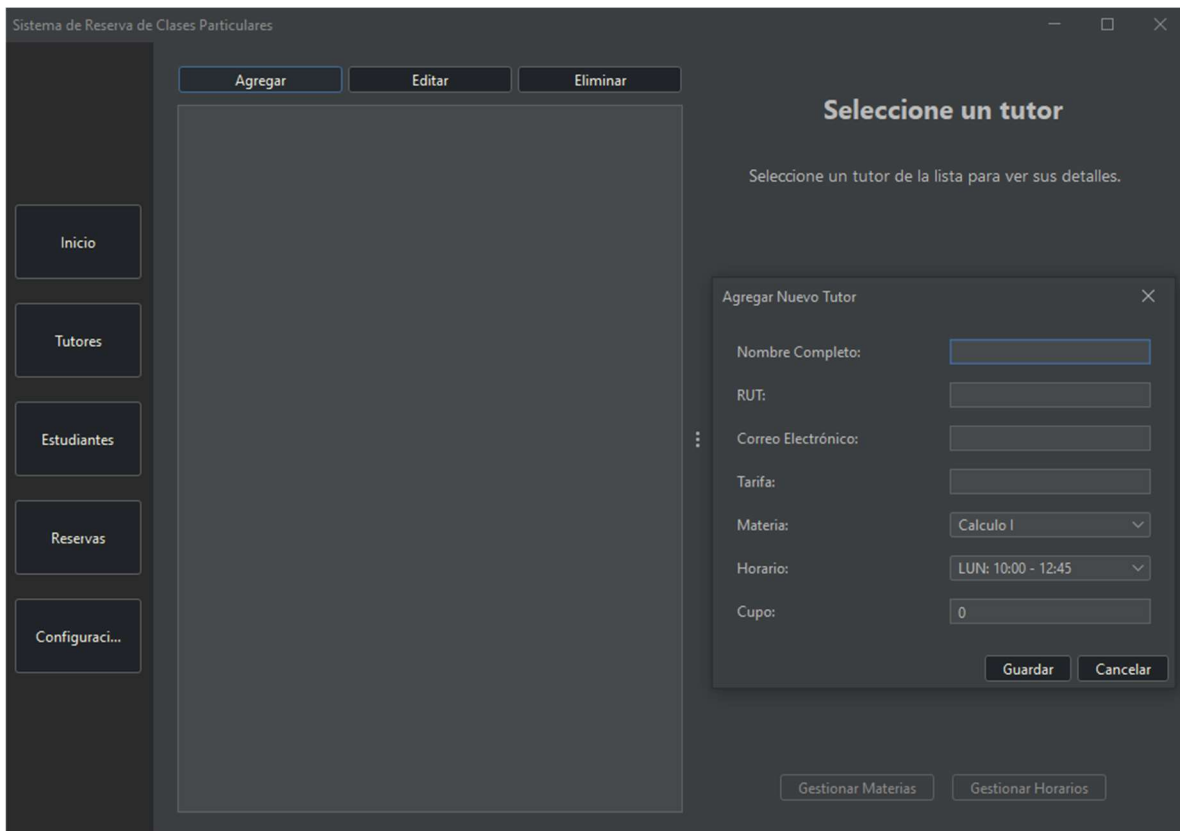
Enunciado

Este sistema está diseñado como una herramienta interna para que un administrador gestione eficientemente las reservas de clases particulares o tutorías. El administrador podrá crear y mantener perfiles detallados de los tutores, incluyendo las materias que imparten, sus tarifas, la cantidad máxima de estudiante por materias y, sus bloques de horarios de disponibilidad. De igual forma, el administrador registrará y gestionará la información de los estudiantes que solicitan el servicio. Cuando se reciba una solicitud de clase el administrador utilizará el sistema para buscar horarios y tutores compatibles con las necesidades del estudiante. Una vez encontrada una opción adecuada, el administrador creará la reserva directamente en el sistema, asignando al estudiante con el tutor en el horario específico. El sistema deberá prevenir conflictos horarios y mantener un "calendario" de las clases programadas con vistas filtradas para cada tutor u estudiante. Además, el administrador se encargará de procesar modificaciones o cancelaciones de clases.

Diagrama de casos de uso



Capturas de la interfaz



Sistema de Reserva de Clases Particulares

Inicio

Tutores

Estudiantes

Reservas

Configuraci...

Filtrar Reservas

Estudiante: Todos

Tutor: Todos

Nueva Reserva

Limpiar Filtros

Fecha	Horario	Materia	Estudiante	Tutor	Estado
-------	---------	---------	------------	-------	--------

Crear Nueva Reserva

Estudiante:

Materia: Calculo I

Horario: LUN: 10:00 - 12:45

Tutor Compatible: Ningún tutor disponible

Modalidad: VIRTUAL

Plataforma:

Enlace:

Fecha (dd-MM-yyyy): 08-07-2026

Crear Reserva

Cancelar

Confirmar Reserva

Cancelar Reserva

Sistema de Reserva de Clases Particulares

Inicio

Tutores

Estudiantes

Reservas

Configuraci...

Gestión de Materias

Calculo I

Calculo II

Calculo III

Física I

Física II

Eliminar

Gestión de Horarios

LUN: 10:00 - 12:45

LUN: 13:00 - 14:45

LUN: 15:00 - 16:45

MAR: 10:00 - 12:45

MAR: 13:00 - 14:45

MAR: 15:00 - 16:45

MIE: 10:00 - 12:45

MIE: 13:00 - 14:45

MIE: 15:00 - 16:45

JUE: 10:00 - 12:45

JUE: 13:00 - 14:45

JUE: 15:00 - 16:45

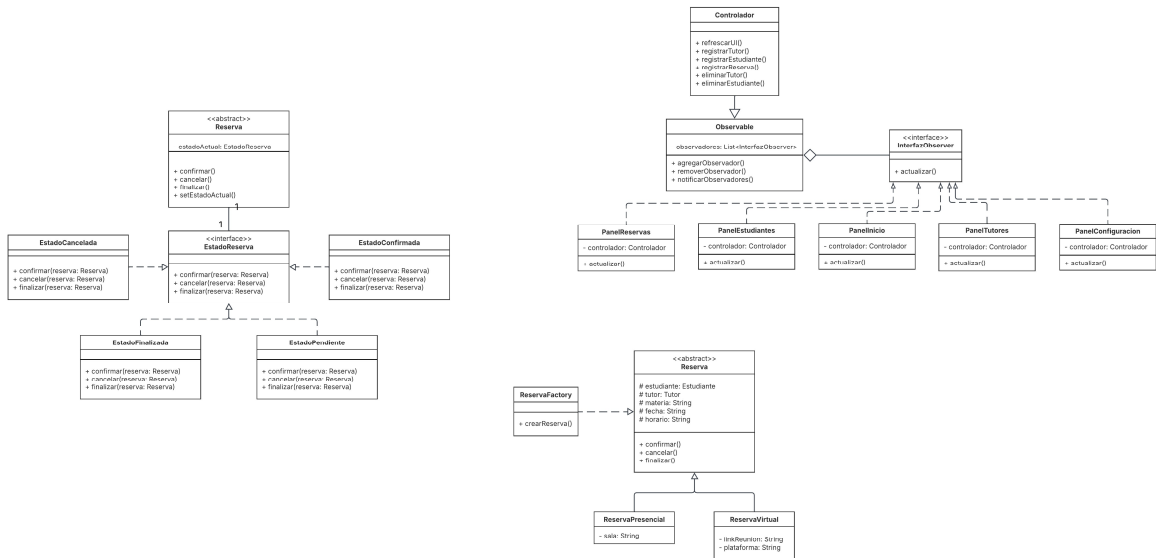
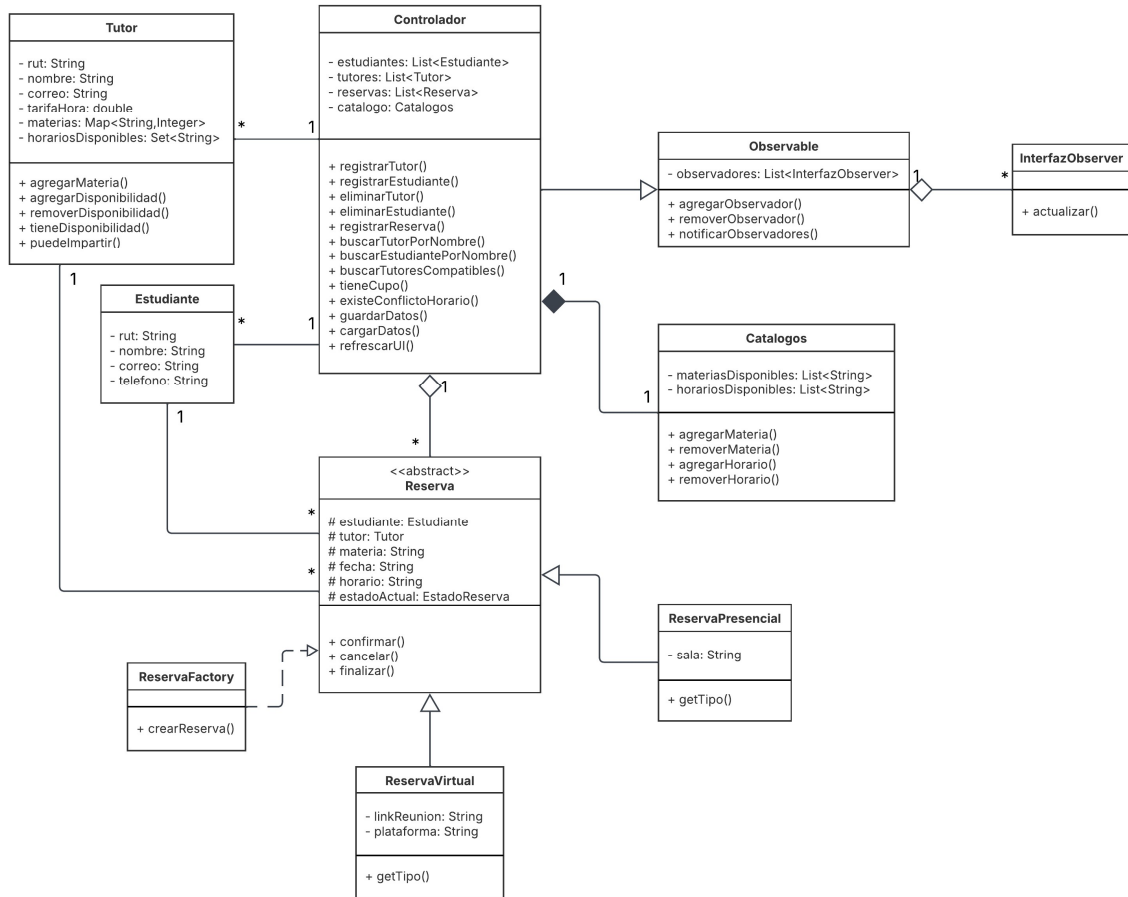
VIE: 10:00 - 12:45

VIE: 13:00 - 14:45

VIE: 15:00 - 16:45

Eliminar

Diagramas de clases UML



Patrones de diseño

Patrón Observer

Se implementó para separar la lógica del programa de la interfaz gráfica. Permite que, al modificar un dato (como agregar un tutor, una reserva o eliminar alguno de estos), todas las vistas se actualicen automáticamente sin tener que escribir código que rompa el patrón MVC.

Clases: Observable, InterfazObserver, Controlador, PanelReservas, PanelTutores, PanelEstudiantes, PanelInicio, PanelConfiguración

Patrón State

Se implementó para gestionar el ciclo de vida de las reservas. En vez de utilizar ifs anidados para saber si una acción que se podía ejecutar sobre una reserva estaba permitida, el patrón se encarga de bloquearlas o permitir las, por ejemplo: intentar cancelar una reserva que ya finalizó.

Clases: EstadoReserva(Interfaz), EstadoPendiente, EstadoConfirmada, EstadoCancelada, EstadoFinalizada, Reserva.

Patrón FactoryMethod

Se implementó para encapsular la creación de reservas. Dependiendo si se seleccionó VIRTUAL o PRESENCIAL, el patrón determina si se debe instanciar una reserva con atributos como sala o si debe hacerlo con atributos como enlace y plataforma. Esto permite mantener la clase controlador limpia de lógica de instanciación.

Clases: ReservaFactory, Reserva(Clase Abstracta), ReservaPresencial, ReservaVirtual

Decisiones Importantes

1. Se tomó la decisión de no permitir la edición de los datos de las reservas una vez creada. Si hay un error se debe cancelar la reserva original y crear una nueva, de esta forma se mantiene la consistencia del patrón State y evita errores al calcular los cupos disponibles de los tutores.
2. En lugar de finalizar una reserva manualmente se implementó una evaluación automática en el controlador. Compara el reloj del sistema con la hora y fecha de la reserva para cambiar el estado a “Finalizada”.
3. Al crear una reserva y seleccionar una materia y un horario, el sistema sugiere automáticamente los tutores que son compatibles con esos parámetros, eliminando la necesidad de buscar de forma manual.
4. Se optó por realizar una colección de horarios y materias, para disminuir el error al ingresar en los campos de datos.

Problemas Identificados

Diseño de interfaz

El desarrollo de una interfaz que sea estética, fácil de navegar y coherente en su función no es simple. Aprender a aplicar *layouts* nuevos y que cumplen la función que se necesitaba consumió tiempo. Además, la cantidad de datos que se necesitaba ingresar era alta (sobre todo en las reservas y tutores) lo que llevó a una saturación de ventanas, haciendo que las vistas se vieran desordenadas y confusas. Como autocrítica, esto sucedió debido a una falta de investigación de todos los *layouts* que existían y sus funciones, lo que nos habría ahorrado mucho tiempo.

Control de errores

Al inicio el programa era muy susceptible a errores del usuario al ingresar datos. Por ejemplo, si alguien intentaba guardar una reserva sin seleccionar un tutor válido, el programa colapsaba, por lo que se tuvieron que agregar muchas verificaciones para asegurarse de que el usuario no pudiera ingresar datos erróneos. Esto sucedió porque al principio programamos asumiendo que el usuario siempre ingresaría los datos correctamente, por lo que tuvimos que retroceder y añadir validaciones y alertas de errores.